Universidad de Cundinamarca

REDES NEURONALES V

Reconocimiento facial con Redes Neuronales Siamesas

Embeddings · Triplet Loss · One-shot y Few-shot Learning · Práctica con fotografías reales

Estudiante: Juan Nicolás Romero Estupiñan

Docente: Nidia Stella Garcia Roa

Noviembre de 2026

1. Introducción y objetivos

El reconocimiento facial mediante aprendizaje profundo no se plantea como un problema de clasificación con un número fijo de clases, sino como un problema de aprendizaje de métricas: una red neuronal transforma cada imagen en un vector de características (embedding) de modo que las imágenes de una misma persona queden cercanas y las de personas distintas queden lejanas. Las Redes Neuronales Siamesas implementan esta idea al procesar varias imágenes con la misma red y los mismos pesos.

Este informe documenta la práctica «PracticaFotosRNV», desarrollada en Google Colab con PyTorch, y sigue la guía «Fotos_RNV». La práctica tiene dos partes: (a) el entrenamiento de un encoder siamés con Triplet Loss sobre un dataset facial sintético de cuatro identidades, y (b) la evaluación de ese encoder con seis fotografías reales de dos personas, mediante distancia euclidiana, similitud coseno y clasificación Few-shot basada en prototipos.

Objetivo general

Implementar y evaluar una red neuronal siamesa para la comparación y verificación de rostros mediante embeddings y Triplet Loss.

Objetivos específicos

- Explicar el funcionamiento de las redes siamesas, los embeddings faciales y Triplet Loss.
- Construir un dataset facial y aplicar preprocesamiento a las imágenes.
- Entrenar un encoder convolucional con Triplet Loss y analizar la separación de distancias intra e inter-identidad.
- Evaluar escenarios One-shot y Few-shot, y verificación de pares con un umbral de distancia.
- Probar el modelo con fotografías reales, interpretar los resultados y reconocer sus limitaciones.

2. Marco conceptual

2.1 Redes Neuronales Siamesas

Una red siamesa usa una misma subred (encoder) con pesos compartidos para procesar dos o más entradas. Cada imagen se proyecta a un espacio de embeddings donde se mide la similitud mediante una distancia. Al compartir pesos, todas las imágenes se representan con la misma función, lo que hace comparables sus embeddings. En este trabajo se emplea la variante con tres ramas (anchor, positive, negative) asociada a Triplet Loss.

2.2 Embeddings faciales y comparación

Un embedding es un vector de dimensión fija (aquí 64) que resume la imagen. Se normaliza a norma L2 unitaria, de manera que los embeddings residen en una hiperesfera. Con vectores normalizados, la distancia euclidiana y la similitud coseno están relacionadas por $d^2 = 2 - 2\cdot\cos(\theta)$, por lo que ambas métricas ordenan los pares de la misma forma.

2.3 Triplet Loss

Cada muestra de entrenamiento es una tripleta (a, p, n): un ancla, un positivo de la misma identidad y un negativo de otra identidad. La función de pérdida exige que el ancla esté más cerca del positivo que del negativo por al menos un margen α:

$$L(a, p, n) = max(||f(a) - f(p)||_2 - ||f(a) - f(n)||_2 + \alpha , 0)$$

Cuando la tripleta ya cumple el margen, la pérdida es cero y no aporta gradiente. En la práctica se utilizó nn.TripletMarginLoss(margin=0.8, p=2).

2.4 One-shot, Few-shot y Zero-shot Learning

Enfoque	Definición	Aplicación en la práctica
One-shot	Reconocer una identidad a partir de un único ejemplo de referencia.	Una imagen de referencia por identidad sintética; los 44 restantes son consultas.
Few-shot	Reconocer con pocos ejemplos por clase, por ejemplo, promediando sus embeddings en un prototipo.	Tres referencias por identidad sintética (36 consultas); dos referencias por persona real (una consulta por persona).
Zero-shot	Reconocer o comparar clases que el modelo no vio durante el entrenamiento y sin ejemplos propios de ellas.	Las fotografías reales y sus identidades no estuvieron en el entrenamiento; el encoder se usó sin ajuste fino (transferencia). Es una interpretación amplia: no se usan descriptores semánticos.

2.5 Importancia de la calidad del dataset

El desempeño de un sistema facial depende de la calidad, variedad y organización del dataset: iluminación, pose, expresión, resolución y fondo. Un dataset poco variado produce embeddings que no generalizan, como se evidencia en la sección 8. Además, las imágenes de rostros son datos personales, por lo que deben usarse con consentimiento explícito y solo con fines académicos.

3. Dataset y consentimiento

3.1 Dataset sintético de entrenamiento

Para entrenar sin comprometer datos personales se generó un dataset sintético con la función synthetic_face(): rostros esquemáticos en escala de grises de 64×64 píxeles, con cuatro identidades (N_ID = 4) y 12 variantes por identidad (N_PER = 12), para un total de 48 imágenes. Cada identidad se distingue por la separación de los ojos, el tamaño de los ojos, la forma de la boca y un rasgo propio (cinta en la frente, barba, línea en el rostro). Las variantes introducen jitter aleatorio de posición (±2 px) y ruido gaussiano (σ = 3).

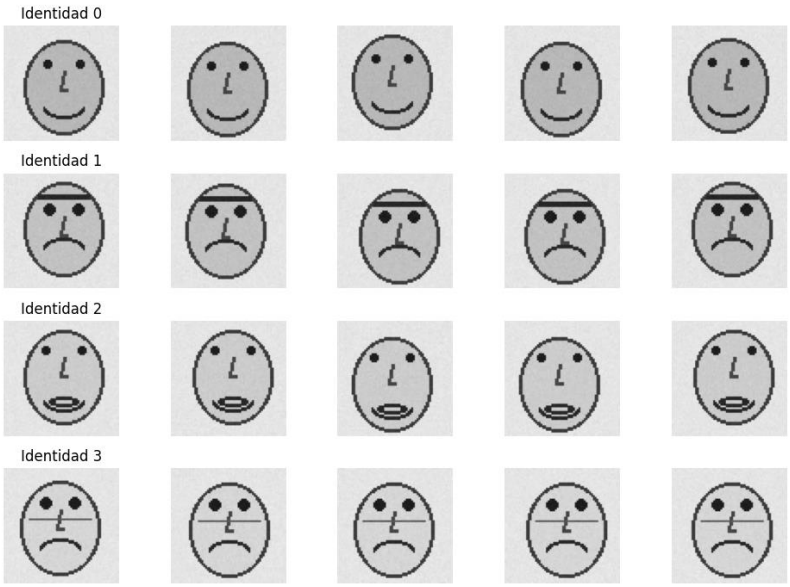


Figura 1. Cinco variantes de cada una de las cuatro identidades sintéticas (Identidad 0 a 3).

3.2 Fotografías reales

Para la segunda parte se cargaron seis fotografías reales: tres de la Persona 1 (persona_01) y tres de la Persona 2 (persona_02), organizadas en carpetas por persona dentro de /content/rostros. Las dos primeras de cada persona son referencias y la tercera es de prueba.

Persona	Foto 1	Foto 2	Foto 3
Persona 1	Referencia	Referencia	Prueba
Persona 2	Referencia	Referencia	Prueba

Consentimiento. Las personas que aparecen en las seis fotografías son personas famosas y las imágenes fueron tomadas de internet. Se emplean exclusivamente con fines académicos, para evaluar el comportamiento del modelo, y no se usan para identificar a nadie ni se publican fuera de este trabajo.

4. Preprocesamiento de las imágenes

El preprocesamiento deja todas las imágenes con el mismo formato y escala, reduce la variabilidad irrelevante y las convierte en tensores que la red puede procesar.

Paso	Descripción
Lectura y conversión a gris	Image.open(...).convert('L'): un solo canal, lo que reduce la dimensionalidad y la dependencia del color.
Ajuste a 64×64	ImageOps.fit(imagen, (64, 64)): recorta al centro y redimensiona manteniendo la proporción, evitando deformar el rostro. La versión inicial (función load_authorized_faces) usa resize directo.
Conversión a tensor	torch.tensor(..., dtype=float32).unsqueeze(1): forma (N, 1, 64, 64).

Paso	Descripción
Normalización de intensidad	División entre 255: valores en [0, 1].
Normalización del embedding	F.normalize(z, p=2, dim=1): norma L2 unitaria a la salida del encoder.

Se dejó la reproducibilidad controlada con SEED = 42 en random, NumPy y PyTorch. No se aplicó detección ni alineación de rostro: el recorte central asume que el rostro está razonablemente centrado, lo cual se discute como limitación en la sección 9.

5. Arquitectura del modelo

El encoder (clase Encoder) es una red convolucional ligera que transforma una imagen (1×64×64) en un embedding de 64 dimensiones con norma unitaria. Es el bloque compartido de la red siamesa: la misma instancia procesa anchor, positive y negative.

#	Capa	Salida (aprox.)	Parámetros
1	Conv2d(1→8, 3×3, pad 1) + ReLU	8×64×64	80
2	MaxPool2d(2)	8×32×32	—
3	Conv2d(8→16, 3×3, pad 1) + ReLU	16×32×32	1 168
4	MaxPool2d(2)	16×16×16	—
5	Conv2d(16→32, 3×3, pad 1) + ReLU	32×16×16	4 640
6	AdaptiveAvgPool2d(4×4) + Flatten	512	—
7	Linear(512→64)	64	32 832
8	Normalización L2	64 (norma 1)	—

El total es de unos 38 700 parámetros, un modelo deliberadamente pequeño que entrena en CPU en segundos (el entorno reportó PyTorch 2.11.0+cpu). AdaptiveAvgPool2d hace que el encoder acepte tamaños de entrada variables antes del flatten.

Código 1. Encoder siamés

```
class Encoder(nn.Module):

  def __init__(self, emb_dim=64):
    super().__init__()

    self.features = nn.Sequential(
      nn.Conv2d(1, 8, 3, padding=1),
      nn.ReLU(),
      nn.MaxPool2d(2),

      nn.Conv2d(8, 16, 3, padding=1),
      nn.ReLU(),
      nn.MaxPool2d(2),
```

```
        nn.Conv2d(16, 32, 3, padding=1),
        nn.ReLU(),
        nn.AdaptiveAvgPool2d((4, 4))
    )

    self.fc = nn.Linear(32 * 4 * 4, emb_dim)

  def forward(self, x):
    z = self.features(x)
    z = z.flatten(1)
    z = self.fc(z)
    z = F.normalize(z, p=2, dim=1)
    return z
```

Salida obtenida:

```
Embedding de ejemplo: torch.Size([2, 64])
```

6. Implementación de Triplet Loss y entrenamiento

6.1 Generación de tripletas

La clase TripletFaces (subclase de Dataset) genera 400 tripletas por época de forma aleatoria: elige una clase ancla y una clase negativa distinta; toma dos imágenes distintas de la clase ancla (anchor y positive) y una imagen de la clase negativa.

Código 2. Dataset de tripletas

```
class TripletFaces(Dataset):
    def __init__(self, X, y, triplets_per_epoch=400):
        self.X = X
        self.y = y
        self.n = triplets_per_epoch

        self.by_class = {
            int(c): torch.where(y == c)[0].tolist()
            for c in torch.unique(y)
        }
        self.classes = list(self.by_class)

    def __len__(self):
        return self.n

    def __getitem__(self, idx):
        a_class = random.choice(self.classes)

        other_classes = [
            c for c in self.classes
            if c != a_class
        ]
        n_class = random.choice(other_classes)

        a_idx, p_idx = random.sample(
            self.by_class[a_class], 2
        )
        n_idx = random.choice(
            self.by_class[n_class]
        )
```

```
        return (
            self.X[a_idx],
            self.X[p_idx],
            self.X[n_idx],
            a_class,
            n_class
        )
```

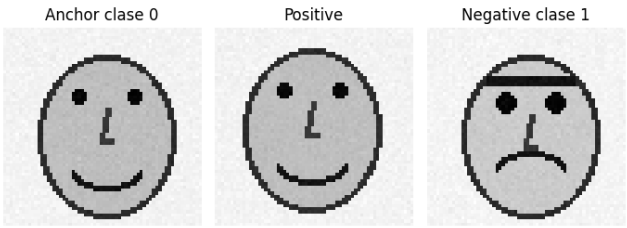


Figura 2. Tripleta de ejemplo: anchor, positive (misma clase) y negative (otra clase).

6.2 Hiperparámetros y bucle de entrenamiento

Hiperparámetro	Valor
Pérdida	nn.TripletMarginLoss, margin = 0.8, p = 2
Optimizador	Adam, lr = 1e-3
Batch size	32
Tripletas por época	400 (≈ 13 lotes)
Épocas	18
Dispositivo	CPU
Semilla	42

Código 3. DataLoader, función de pérdida y optimizador

```
loader = DataLoader(
  trip,
  batch_size=32,
  shuffle=True
)

criterion = nn.TripletMarginLoss(
  margin=0.8,
  p=2
)

optimizer = torch.optim.Adam(
  model.parameters(),
  lr=1e-3
)
```

Código 4. Bucle de entrenamiento

```
losses = []

for epoch in range(18):
    model.train()
    total = 0
```

```
    count = 0

    for a, p, n, _, _ in loader:
        a = a.to(DEVICE)
        p = p.to(DEVICE)
        n = n.to(DEVICE)

        za = model(a)
        zp = model(p)
        zn = model(n)

        loss = criterion(za, zp, zn)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total += loss.item() * a.size(0)
        count += a.size(0)

    mean_loss = total / count
    losses.append(mean_loss)

    if epoch in [0, 1, 2, 5, 10, 17]:
        print(
            f'Época {epoch+1:02d} | '
            f'loss={mean_loss:.4f}'
        )
```

Salida obtenida:

```
Época 01 | loss=0.3981
Época 02 | loss=0.0266
Época 03 | loss=0.0024
Época 06 | loss=0.0000
Época 11 | loss=0.0002
Época 18 | loss=0.0000
```

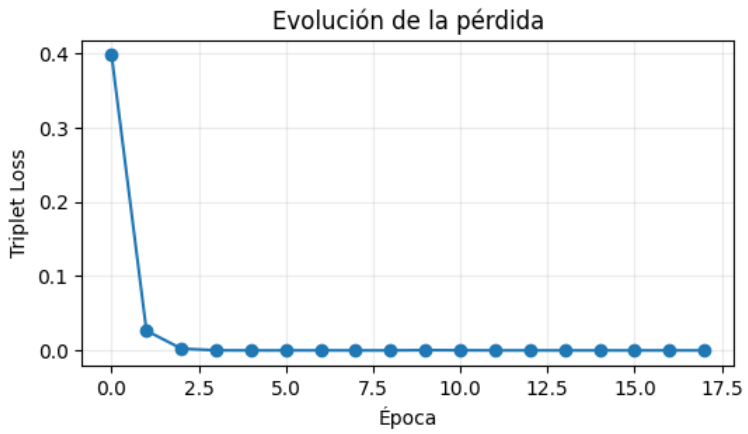


Figura 3. Evolución de la Triplet Loss durante las 18 épocas.

La pérdida cae de 0.398 en la primera época a valores prácticamente nulos desde la sexta. Este rápido descenso indica que la tarea sintética es fácil: las identidades se diferencian por rasgos geométricos muy marcados y el margen de 0.8 se cumple pronto.

7. Resultados con el dataset sintético

7.1 Distancias entre embeddings

Se generaron embeddings para las 48 imágenes y se calcularon las distancias euclidianas entre todos los pares (1 128 pares).

Medida	Valor
Distancia media, misma identidad	0.1312
Distancia media, identidad diferente	1.5421

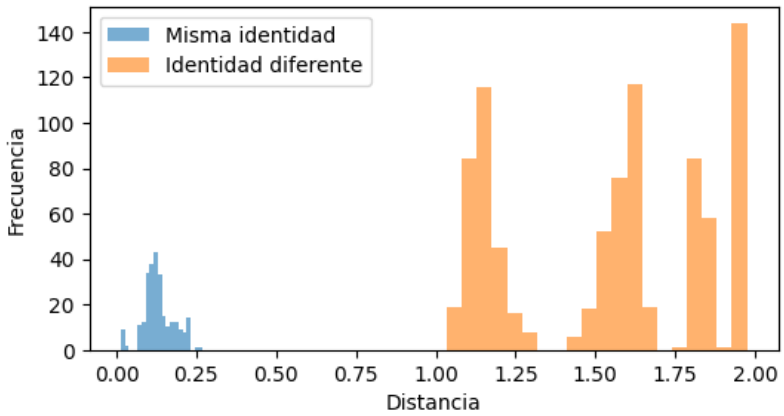


Figura 4. Histograma de distancias: misma identidad (azul) frente a identidades diferentes (naranja).

Las dos distribuciones están totalmente separadas: las distancias intra-identidad no superan ≈ 0.25 y las inter-identidad comienzan en ≈ 1.0. La separación confirma que el encoder aprendió una métrica coherente en este dominio.

7.2 Reconocimiento One-shot y Few-shot

Código 5. Evaluación One-shot (1 referencia por identidad)

```
refs = {
  c: np.where(labels == c)[0][0]
  for c in range(N_ID)
}

queries = [
  i for i in range(len(X))
  if i not in refs.values()
]

correct = 0

for i in queries:
  dists = {
    c: torch.dist(
    Z[i], Z[idx]
    ).item()
    for c, idx in refs.items()
  }
```

```
  pred = min(dists, key=dists.get)
  correct += int(pred == labels[i])

acc_one = correct / len(queries)

print(
  'Accuracy One-shot:',
  round(acc_one, 4),
  f'({correct}/{len(queries)})'
)
```

Código 6. Evaluación Few-shot con prototipos (3 referencias por identidad)

```
ref_ids = {
  c: np.where(labels == c)[0][:3]
  for c in range(N_ID)
}
prototypes = {}
for c, idx in ref_ids.items():
  proto = Z[idx].mean(
    0, keepdim=True
  )
  proto = F.normalize(
    proto, p=2, dim=1
  )
  prototypes[c] = proto.squeeze(0)

query_ids = [
  i for i in range(len(X))
  if all(
    i not in ids
    for ids in ref_ids.values()
  )
]

correct = 0

for i in query_ids:
  dists = {
    c: torch.dist(
      Z[i], proto
    ).item()
    for c, proto in prototypes.items()
  }

  pred = min(dists, key=dists.get)
  correct += int(pred == labels[i])

acc_few = correct / len(query_ids)

print(
  'Accuracy Few-shot:',
  round(acc_few, 4),
  f'({correct}/{len(query_ids)})'
)
print(
  'Comparación -> One-shot:',
  round(acc_one, 4),
  '| Few-shot:',
  round(acc_few, 4)
)
```

Escenario	Referencias	Consultas	Exactitud
One-shot	1 por identidad	44	100 % (44/44)
Few-shot (prototipo)	3 por identidad	36	100 % (36/36)

7.3 Verificación de pares con umbral

Se definió un umbral de decisión como el punto medio entre las distancias medias positiva y negativa: (0.1312 + 1.5421) / 2 = 0.8367. Un par se declara «misma identidad» si su distancia es menor que el umbral.

Código 7. Verificación de pares

```
import numpy as np
import torch


threshold = (
  float(np.mean(pos)) +
  float(np.mean(neg))
) / 2

print(
  'Umbral pedagógico:',
  round(threshold, 4)
)

def verify(i, j):
  distance = torch.dist(
    Z[i], Z[j]
  ).item()

  same_identity = distance < threshold
  return distance, same_identity

same = np.where(labels == 0)[0][:2]

diff = (
  np.where(labels == 0)[0][0],
  np.where(labels == 1)[0][0]
)

for pair in [tuple(same), diff]:
  distance, ok = verify(*pair)
  result = 'MISMA' if ok else 'DIFERENTE'

  print(
  pair,
  'dist=', round(distance, 4),
  '=>', result
  )
```

Par	Distancia	Decisión	Correcto
(0, 1): misma identidad 0	0.2070	MISMA	Sí
(0, 12): identidades 0 y 1	1.2789	DIFERENTE	Sí

Nota metodológica. Los embeddings se evaluaron sobre las mismas 48 imágenes con las que se muestrearon las tripletas de entrenamiento, y el umbral se calculó con esas mismas distancias. No hubo conjunto de validación independiente, por lo que el 100 % de exactitud mide el ajuste al dataset sintético y no la capacidad de generalizar.

8. Práctica con fotografías reales

8.1 Carga y organización

Las fotografías se subieron con google.colab.files.upload() a dos carpetas (/content/rostros/persona_01 y /content/rostros/persona_02), y se verificó la estructura: 6 archivos en total, 3 por persona. Luego se aplicó el preprocesamiento de la sección 4 con la función cargar_rostros(), que produjo un tensor de forma (6, 1, 64, 64) y etiquetas [0, 0, 0, 1, 1, 1].

Código 8. Carga y preprocesamiento de fotografías reales

```
def cargar_rostros(carpeta='/content/rostros', tamano=(64, 64)):
    carpeta = Path(carpeta)

    imagenes = []
    etiquetas = []
    nombres_personas = []

    personas = sorted([p for p in carpeta.iterdir() if p.is_dir()])

    for etiqueta, persona in enumerate(personas):
        nombres_personas.append(persona.name)
        print('Leyendo:', persona.name)

        for archivo in sorted(persona.glob('*')):
            if archivo.suffix.lower() not in {'.jpg', '.jpeg', '.png', '.webp'}:
                continue

            imagen = Image.open(archivo).convert('L')
            imagen = ImageOps.fit(imagen, tamano)
            imagen = np.array(imagen, dtype=np.uint8)

            imagenes.append(imagen)
            etiquetas.append(etiqueta)

    Xr = torch.tensor(np.stack(imagenes), dtype=torch.float32)
    Xr = Xr.unsqueeze(1) / 255.0

    yr = torch.tensor(etiquetas, dtype=torch.long)

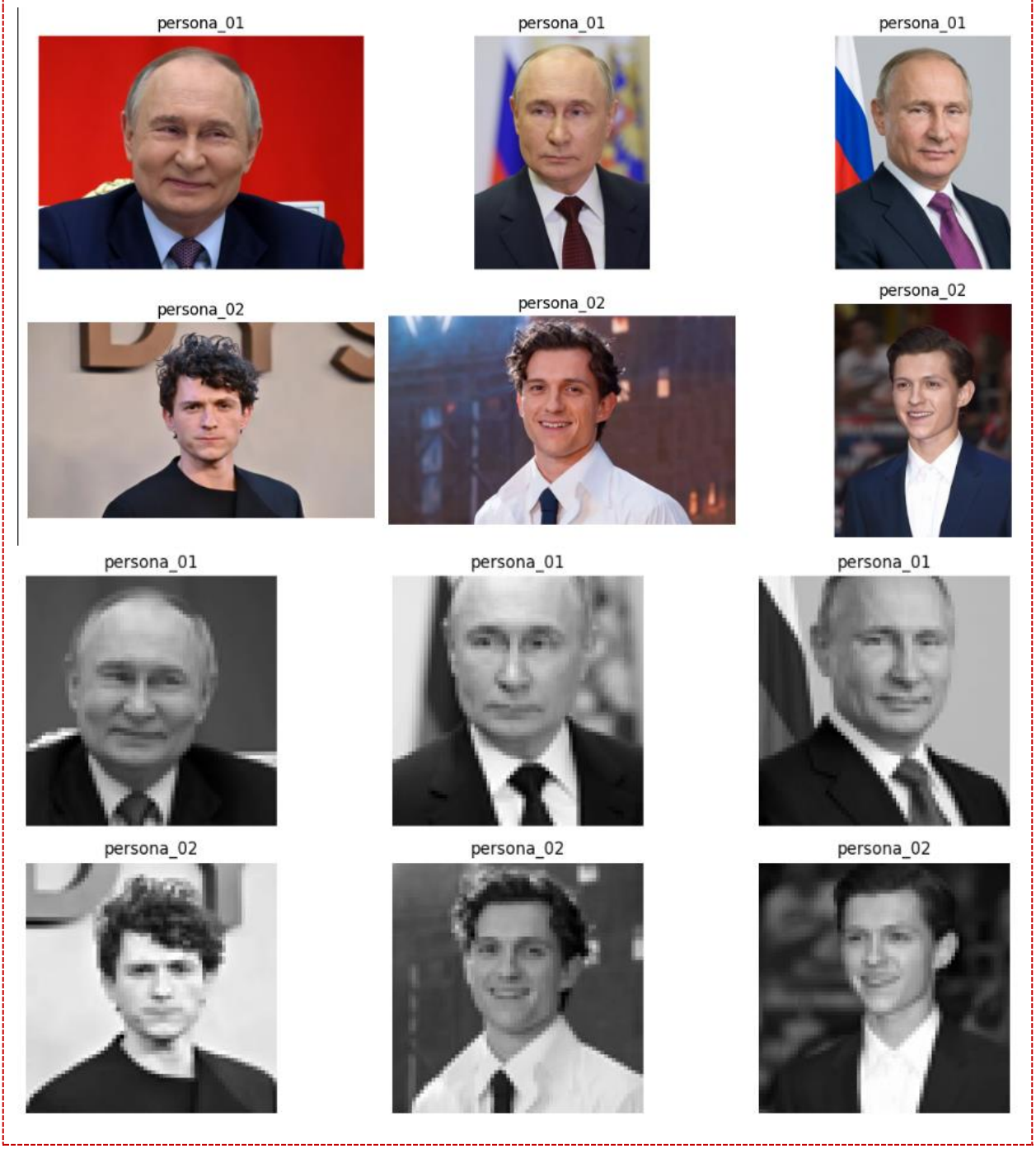
    return Xr, yr, nombres_personas
```

Salida obtenida:

```
Carpeta: /content/rostros
Carpeta: /content/rostros/persona_01
   foto3.jpg  foto1.jpg  foto2.jpg
Carpeta: /content/rostros/persona_02
   foto2 (1).jpg  foto1 (1).jpg  foto3 (1).jpg
Total de archivos encontrados: 6
```

Salida obtenida:

```
Personas encontradas: ['persona_01', 'persona_02']
Forma de las imágenes: torch.Size([6, 1, 64, 64])
Etiquetas: tensor([0, 0, 0, 1, 1, 1])
```



8.2 Triple real: anchor, positive y negative

Se seleccionó como anchor la primera foto de la Persona 1 (índice 0), como positive la segunda foto de la Persona 1 (índice 1) y como negative la primera foto de la Persona 2 (índice 3). El encoder entrenado generó un embedding de 64 dimensiones para cada una.



8.3 Distancia euclidiana y similitud coseno

Código 9. Distancia euclidiana entre embeddings normalizados

```
distancia_AP = torch.dist(
    embedding_anchor_norm,
    embedding_positive_norm
).item()

distancia_AN = torch.dist(
    embedding_anchor_norm,
    embedding_negative_norm
).item()

print('Distancia misma persona:', round(distancia_AP, 4))
print('Distancia personas diferentes:', round(distancia_AN, 4))
```

Medida	Misma persona (A–P)	Personas distintas (A–N)
Distancia euclidiana	0.2635	0.4677
Similitud coseno	0.9653	0.8907

La red acerca correctamente las imágenes de la misma persona (0.2635 < 0.4677). Sin embargo, la diferencia es pequeña: a diferencia del dominio sintético, donde los pares negativos estaban a ≈ 1.5, los pares de personas distintas aparecen a solo 0.47, y su similitud coseno sigue siendo alta (0.89). Verificación consistente: para vectores unitarios $d^2 = 2 - 2\cdot\cos$, y $0.2635^2 = 0.0694 \approx 2 - 2\cdot0.9653$.

8.4 Prototipos y clasificación Few-shot

Para cada persona se promediaron los embeddings de sus dos fotos de referencia y se renormalizó el resultado para obtener un prototipo. Cada foto restante (la tercera de cada persona) se clasificó asignándola al prototipo con mayor similitud coseno.

Código 10. Prototipo de la Persona 1

```
referencias_p1 = embeddings[indices_p1[:2]]

prototipo_p1 = referencias_p1.mean(
    dim=0,
    keepdim=True
)

prototipo_p1 = F.normalize(
    prototipo_p1,
    p=2,
    dim=1
)

print('Prototipo Persona 1:', prototipo_p1.shape)
```

Código 11. Clasificación de la foto de prueba de la Persona 1

```
consulta_p1 = embeddings[indices_p1[2]].unsqueeze(0)

sim_p1_p1 = F.cosine_similarity(
    consulta_p1,
    prototipo_p1
).item()

sim_p1_p2 = F.cosine_similarity(
    consulta_p1,
    prototipo_p2
).item()

print('FOTO REAL: Persona 1')
print('Similitud con Persona 1:', round(sim_p1_p1, 4))
print('Similitud con Persona 2:', round(sim_p1_p2, 4))

if sim_p1_p1 > sim_p1_p2:
    prediccion_p1 = 'Persona 1'
else:
    prediccion_p1 = 'Persona 2'

print('Predicción:', prediccion_p1)
```

Foto de prueba	Sim. con prototipo P1	Sim. con prototipo P2	Predicción	Resultado
Persona 1, foto 3	0.9554	0.8113	Persona 1	Correcto
Persona 2, foto 3	0.8903	0.9359	Persona 2	Correcto

Exactitud Few-shot sobre fotografías reales: 2 de 2 (100 %). Los márgenes entre similitudes son 0.144 para la Persona 1 y solo 0.046 para la Persona 2, es decir, la segunda decisión fue mucho más ajustada.

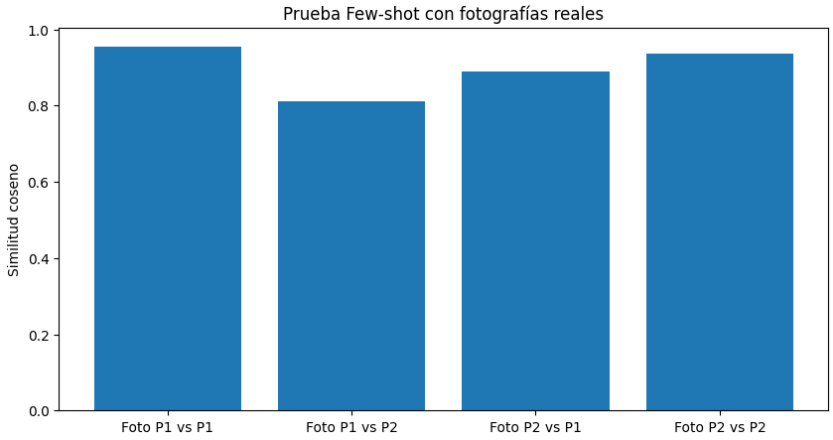


Figura 5. Similitud coseno de cada foto de prueba frente a los prototipos de ambas personas.

8.5 Matriz de similitud entre las seis fotografías

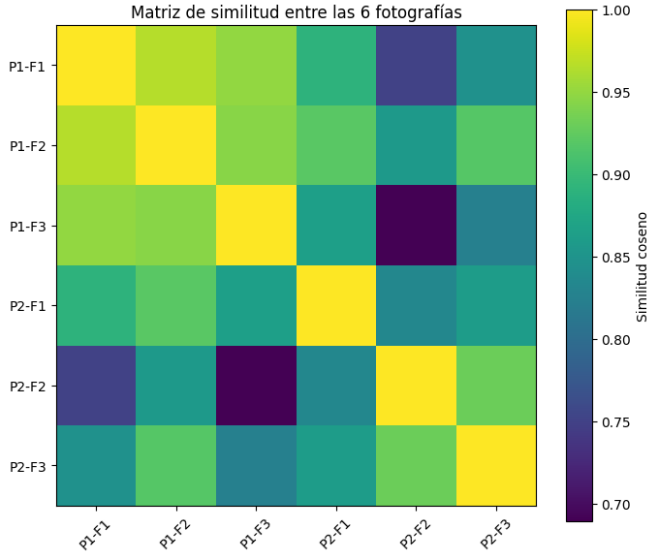


Figura 6. Matriz de similitud coseno entre las seis fotografías.

Las similitudes entre fotos de la misma persona están en torno a 0.93–0.96, y las de personas distintas entre 0.69 y 0.93. Hay pares de personas distintas con similitud alta (por ejemplo P1-F2 con P2-F2, ≈ 0.92), lo que muestra que las clases no están claramente separadas: los rangos de similitud se solapan.

9. Análisis de resultados y limitaciones

- **Contraste entre dominios.** En el dataset sintético la distancia media negativa fue 1.5421 frente a 0.1312 de la positiva (razón ≈ 12). En fotografías reales fue 0.4677 frente a 0.2635 (razón ≈ 1.8). El encoder no conserva la separación al cambiar de dominio.
- **El umbral sintético no transfiere.** El umbral 0.8367 declararía «misma persona» al par negativo real (distancia 0.4677 < 0.8367), es decir, produciría un falso positivo. En fotos reales haría falta calibrar un umbral propio con datos reales.

- **Muestra mínima.** Con dos consultas reales, el 100 % de exactitud no es estadísticamente concluyente: un solo error cambiaría el resultado a 50 %.
- **Entrenamiento sobre rostros esquemáticos.** El encoder nunca vio texturas, iluminación, poses ni fondos reales. Que clasifique bien 2 de 2 puede deberse a diferencias generales de intensidad y encuadre de las fotos, no necesariamente a rasgos faciales de identidad.
- **Preprocesamiento.** No se detecta ni alinea el rostro; el recorte central a 64×64 en escala de grises puede incluir fondo, ropa y bandas de color que afectan el embedding.
- **Evaluación sin partición.** En la parte sintética, entrenamiento y evaluación usan las mismas imágenes (ver nota de la sección 7.3).

Mejoras propuestas

- Usar un modelo preentrenado en reconocimiento facial (por ejemplo, FaceNet o ArcFace) o ajustar el encoder con un dataset facial real y autorizado.
- Detectar y alinear el rostro (MTCNN, MediaPipe o similar) antes del recorte, y aumentar la resolución de entrada (por ejemplo 112×112 o 160×160, en color).
- Aplicar minería de tripletas difíciles (hard / semi-hard negative mining) y aumento de datos (rotación, brillo, recorte).
- Separar el dataset en entrenamiento, validación y prueba, y medir ROC, EER y tasas de falsa aceptación y falso rechazo para calibrar el umbral.
- Aumentar el número de personas y fotografías por persona con consentimiento documentado.

10. Evidencias de ejecución

Se presentan las salidas reales de ejecución del notebook en Google Colab, en el orden de la práctica.

10.1 Entorno y verificación del modelo

Celda 1. Semilla y dispositivo

```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'

print('PyTorch:', torch.__version__)
print('Dispositivo:', DEVICE)
```

Salida obtenida:

```
PyTorch: 2.11.0+cpu
Dispositivo: cpu
```

Celda 19. Comprobación de model y DEVICE

```
print('Modelo disponible:', 'model' in globals())
print('DEVICE disponible:', 'DEVICE' in globals())

if 'model' not in globals() or 'DEVICE' not in globals():
    print('Debes ejecutar primero la práctica principal de Redes Neuronales V.')
```

```
else:
    print('Podemos continuar con fotografías reales.')
```

Salida obtenida:

```
Modelo disponible: True
DEVICE disponible: True
Podemos continuar con fotografías reales.
```

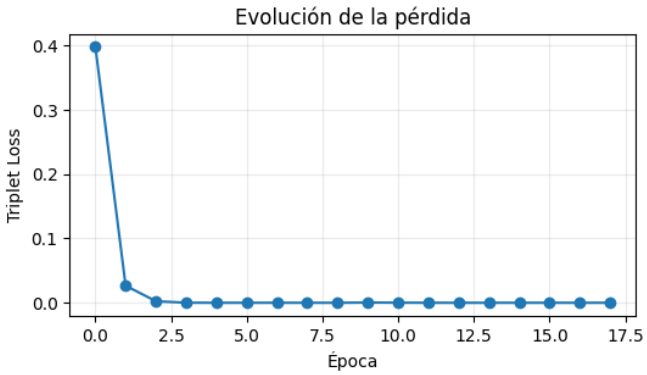
10.2 Arquitectura y entrenamiento

Salida obtenida:

```
Encoder(
  (features): Sequential(
    (0): Conv2d(1, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(8, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU()
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU()
    (8): AdaptiveAvgPool2d(output_size=(4, 4))
  )
  (fc): Linear(in_features=512, out_features=64, bias=True)
)
Embedding de ejemplo: torch.Size([2, 64])
```

Salida obtenida:

```
Época 01 | loss=0.3981
Época 02 | loss=0.0266
Época 03 | loss=0.0024
Época 06 | loss=0.0000
Época 11 | loss=0.0002
Época 18 | loss=0.0000
```



Evidencia 1. Curva de Triplet Loss.

10.3 Pruebas con el dataset sintético

Salida obtenida:

```
Distancia positiva media: 0.1312
Distancia negativa media: 1.5421


Accuracy One-shot: 1.0 (44/44)
```

```
Accuracy Few-shot: 1.0 (36/36)

Umbral pedagógico: 0.8367
(0, 1) dist= 0.207 => MISMA
(0, 12) dist= 1.2789 => DIFERENTE
```

10.4 Pruebas con fotografías reales

Salida obtenida:

```
Embedding Anchor: torch.Size([1, 64])
Embedding Positive: torch.Size([1, 64])
Embedding Negative: torch.Size([1, 64])

Distancia misma persona: 0.2635
Distancia personas diferentes: 0.4677
RESULTADO: La red acerca correctamente las imágenes de la misma persona.

Similitud misma persona: 0.9653
Similitud personas diferentes: 0.8907
```

Salida obtenida:

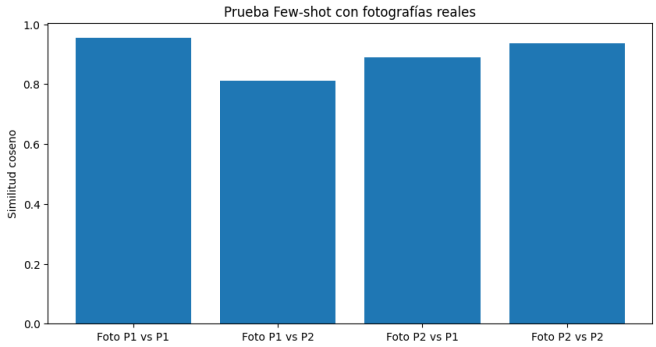
```
FOTO REAL: Persona 1
Similitud con Persona 1: 0.9554
Similitud con Persona 2: 0.8113
Predicción: Persona 1

FOTO REAL: Persona 2
Similitud con Persona 1: 0.8903
Similitud con Persona 2: 0.9359
Predicción: Persona 2
```

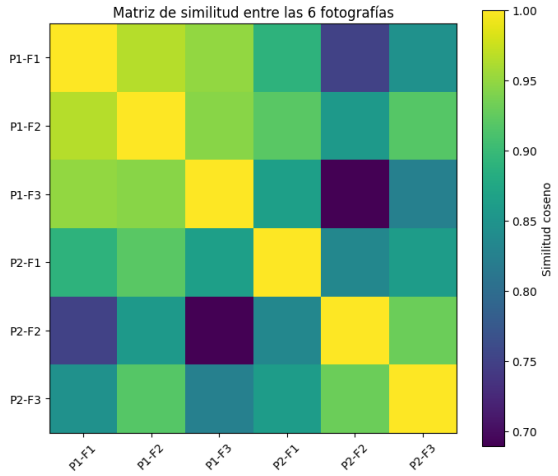
Salida obtenida:

```
==========================================
PRUEBA FEW-SHOT
==========================================
 CASO 1
Real: Persona 1
Predicción: Persona 1
Resultado: CORRECTO
 CASO 2
Real: Persona 2
Predicción: Persona 2
Resultado: CORRECTO

Aciertos: 2 de 2
Precisión: 100.0 %
```



Evidencia 2. Similitud coseno de las pruebas Few-shot.



Evidencia 3. Matriz de similitud entre las seis fotografías.

11. Conclusiones

- Se implementó una red siamesa funcional en PyTorch, con un encoder convolucional compartido y salida de 64 dimensiones normalizada, entrenada con Triplet Loss (margen 0.8) hasta pérdida casi nula en 18 épocas.
- En el dominio sintético el modelo separó por completo identidades distintas (distancia media 1.5421) de la misma identidad (0.1312) y alcanzó 100 % en One-shot y Few-shot; este resultado debe leerse con cautela, porque no hubo partición de datos.
- Con fotografías reales, el encoder acercó las imágenes de la misma persona (distancia 0.2635 frente a 0.4677) y clasificó correctamente 2 de 2 imágenes de prueba mediante prototipos, pero con márgenes pequeños y solapamiento de similitudes entre personas distintas.
- El Few-shot con prototipos y la similitud coseno son una forma sencilla de comparar identidades sin reentrenar, y la normalización L2 vuelve comparables las medidas. La transferencia de un dominio sintético a uno real es limitada, lo que confirma la importancia de la calidad y diversidad del dataset facial.
- El sistema es un prototipo académico, no un reconocedor facial robusto, y su uso exige consentimiento explícito de las personas retratadas.

12. Repositorio GitHub

Enlace al repositorio: https://github.com/NicolasRomero22/PracticaRNVFotos

13. Referencias

- Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A Unified Embedding for Face Recognition and Clustering. arXiv:1503.03832. https://arxiv.org/abs/1503.03832
- Hoffer, E., & Ailon, N. (2014). Deep Metric Learning Using Triplet Network. arXiv:1412.6622. https://arxiv.org/abs/1412.6622
- Snell, J., Swersky, K., & Zemel, R. (2017). Prototypical Networks for Few-shot Learning. arXiv:1703.05175. https://arxiv.org/abs/1703.05175
- PyTorch Documentation. torch.nn.TripletMarginLoss. https://pytorch.org/docs/stable/generated/torch.nn.TripletMarginLoss.html
- PyTorch Documentation. torch.nn.functional.normalize. https://pytorch.org/docs/stable/generated/torch.nn.functional.normalize.html
- PyTorch Documentation. torch.nn.functional.cosine_similarity. https://pytorch.org/docs/stable/generated/torch.nn.functional.cosine_similarity.html
- Google Colaboratory. https://colab.research.google.com
- Guía del estudiante «Redes Neuronales V – Práctica complementaria con fotografías reales» (Fotos_RNV), documento de la actividad.